

“Año de la Esperanza y el Fortalecimiento de la Democracia”

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

(Universidad del Perú, Decana de América)

FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMATICA



INFORME DEL PROYECTO FINAL

SISTEMA DE CONTROL DE INVENTARIO

GRUPO 2

Integrantes:

Bellodas Ramos, Emily Guisell	(24200080)
Bustos Huayanay, Bruno Renato	(24200003)
Carbajo Gallegos, Gael Stephan	(24200092)
Figueroa Estrella, Jhair Alberto	(24200013)
Jimenes Trujillo, Jack Bryan	(24200109)
Peña Torres, Adan Imanol	(24200083)
Saldaña Sanchez, Luis Mario	(24200038)
Vargas Quispe, Sebastian Alexandre	(24200131)
Yucra Tintaya, Anthony Josue	(24200043)

Docente: Lic. Cabrera Díaz, Javier Elmer

Lima – Perú

2026- I

TABLA DE CONTENIDOS

1. Introducción.....	3
2. Objetivos.....	4
2.1 Objetivo general.....	4
2.2 Objetivos específicos.....	4
3. Alcance del sistema.....	5
4. Organización del proyecto.....	6
Figura 1. Estructura del proyecto en NetBeans.....	6
4.1 Paquete estructuras.....	7
4.2 Paquete modelos.....	7
4.3 Paquete servicios.....	8
4.4 Paquete excepciones.....	8
5. Descripción de módulos del sistema.....	9
5.1 Módulo principal.....	9
Figura 2. Ventana principal del sistema.....	9
5.2 Módulo de gestión de productos.....	10
Figura 3. Módulo de gestión de productos.....	10
5.3 Módulo de control de stock.....	10
Figura 4. Módulo de control de stock.....	11
5.4 Módulo de logística.....	11
Figura 5. Módulo de logística.....	12
5.5 Módulo de reportes.....	12
Figura 6. Módulo de reportes.....	13
6. Estructuras de datos implementadas.....	14
6.1 Lista enlazada como estructura auxiliar.....	14
Aplicación en el sistema.....	14
6.2 Árbol Binario de Búsqueda.....	15
Representación conceptual.....	15
Aplicación dentro del sistema.....	15
Código 1. Fragmento del Árbol Binario de Búsqueda.....	16
Complejidad del Árbol Binario de Búsqueda.....	16
6.3 Pila de Auditoría.....	17
Principio LIFO.....	17
Implementación de la pila.....	17
Operaciones principales.....	17
Código 2. Fragmento de la Pila de Auditoría.....	18
Código 3. Fragmento de extracción o consulta de la pila.....	19
Aplicación de la pila en el sistema.....	19
Ventajas de usar una pila.....	19
6.4 Grafo Logístico.....	19

Elementos del grafo.....	20
Grafo no dirigido.....	20
Grafo ponderado.....	20
Lista de adyacencia.....	21
Código 4. Fragmento de conexión entre almacenes.....	21
Operaciones principales del grafo.....	22
Figura 7. Representación de la red logística.....	23
6.5 Algoritmo de Dijkstra.....	23
Funcionamiento general.....	23
Datos utilizados por el algoritmo.....	24
Código 5. Fragmento del algoritmo de Dijkstra.....	24
Resultado de Dijkstra.....	24
Ejemplo conceptual.....	25
Figura 8. Resultado del cálculo de ruta óptima.....	26
Complejidad de Dijkstra.....	26
7. Funcionamiento general del sistema.....	27
7.1 Registro de productos.....	27
Figura 9. Registro exitoso de producto.....	28
7.2 Búsqueda de productos.....	28
Figura 10. Búsqueda de producto.....	29
7.3 Movimiento de stock.....	29
Figura 11. Registro de movimiento de stock.....	29
7.4 Registro de almacenes y rutas.....	29
Figura 12. Registro de almacenes y rutas.....	30
7.5 Cálculo de ruta logística.....	30
Figura 13. Ejecución del cálculo de ruta logística.....	31
8. Manejo de excepciones.....	32
Figura 14. Mensaje de error controlado.....	32
9. Evaluación técnica del proyecto.....	33
10. Posibles mejoras.....	34
11. Conclusiones.....	35
12. Bibliografía.....	36
13. Anexos.....	37
Anexo 1. Fragmentos de código relevantes.....	37
Anexo 1.1 Código del Árbol Binario de Búsqueda.....	37
Anexo 1.2 Código de la Pila de Auditoría.....	37
Anexo 1.3 Código del Grafo Logístico.....	37
Anexo 2. Capturas adicionales del sistema.....	38

1. Introducción

El presente proyecto consiste en el desarrollo de un sistema de control de inventario y logística implementado en el lenguaje de programación Java, utilizando el entorno de desarrollo NetBeans. La aplicación permite administrar productos, controlar movimientos de stock, registrar transacciones, gestionar almacenes y calcular rutas logísticas entre diferentes puntos de almacenamiento.

El sistema fue desarrollado bajo una organización modular, separando las clases en paquetes según su responsabilidad. Esta organización facilita la lectura, mantenimiento y ampliación del código. Asimismo, el proyecto aplica conceptos de programación orientada a objetos, manejo de excepciones personalizadas, interfaces gráficas con Java Swing y estructuras de datos implementadas de forma propia.

Una de las características principales del sistema es el uso de estructuras de datos como el Árbol Binario de Búsqueda, la Pila y el Grafo. Estas estructuras permiten resolver problemas específicos dentro del sistema: el árbol facilita la organización y búsqueda de productos, la pila registra las transacciones de auditoría bajo el principio LIFO y el grafo representa la red logística de almacenes conectados mediante rutas.

El desarrollo del proyecto permitió aplicar conceptos teóricos de estructuras de datos en una solución práctica relacionada con la gestión de inventarios y logística.

2. Objetivos

2.1 Objetivo general

Desarrollar un sistema de escritorio en Java para la gestión de inventario y logística, utilizando estructuras de datos personalizadas que permitan administrar productos, controlar stock, registrar transacciones y calcular rutas entre almacenes.

2.2 Objetivos específicos

- Implementar un módulo para registrar, buscar, modificar y eliminar productos.
- Controlar entradas y salidas de stock dentro del sistema.
- Registrar las operaciones realizadas mediante una pila de auditoría.
- Administrar almacenes dentro de una red logística.
- Representar las rutas entre almacenes mediante un grafo ponderado.
- Calcular la ruta más corta entre almacenes utilizando el algoritmo de Dijkstra.
- Manejar errores del sistema mediante excepciones personalizadas.
- Desarrollar una interfaz gráfica que facilite la interacción del usuario con el sistema.
- Aplicar estructuras de datos en un caso práctico de gestión empresarial.

3. Alcance del sistema

El sistema desarrollado permite realizar operaciones básicas y avanzadas relacionadas con el control automatizado de inventarios y la optimización de procesos logísticos. Su alcance principal abarca la administración detallada de productos, el monitoreo dinámico y alertas de stock, el registro histórico de transacciones, la gestión eficiente de múltiples almacenes, el trazado y conexión de rutas de distribución, y la generación de reportes analíticos para la toma de decisiones.

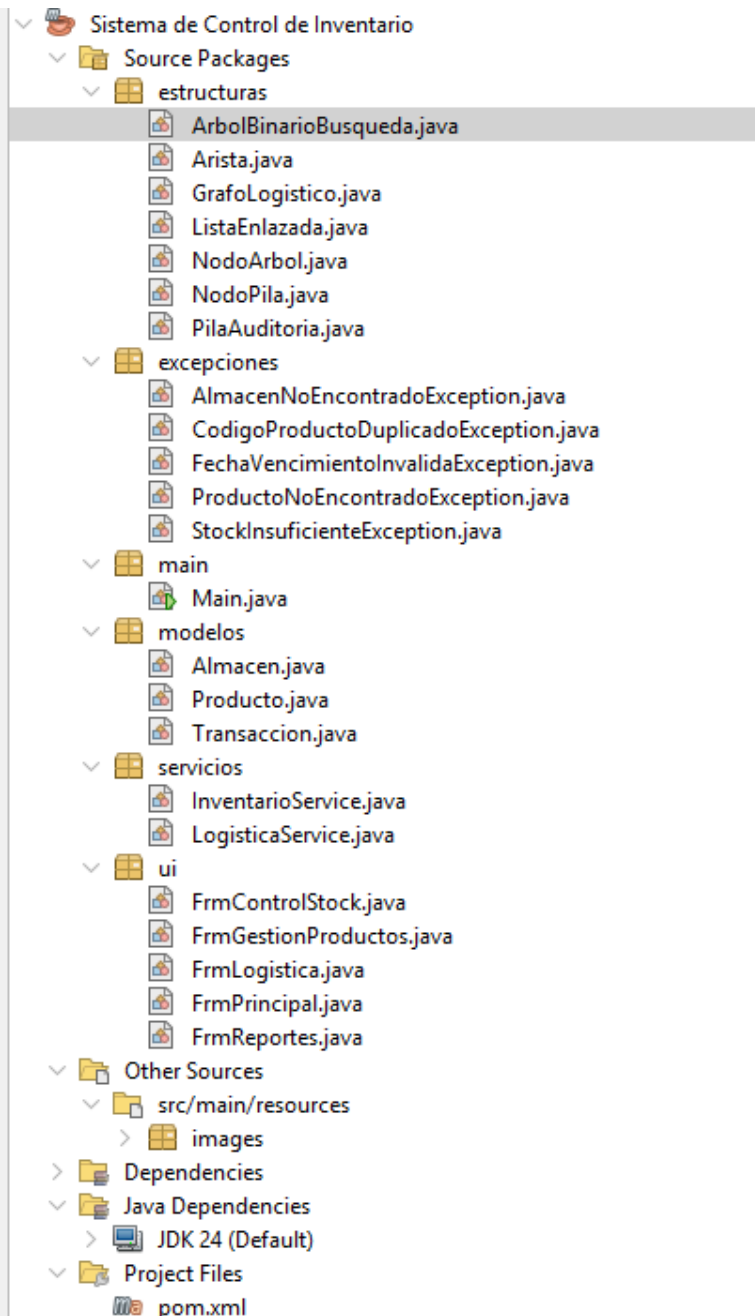
Diseñado bajo un enfoque robusto de aplicación de escritorio, la interacción con el usuario se realiza mediante interfaces gráficas intuitivas desarrolladas en Java Swing. Por su parte, el flujo de la información y la lógica de negocio se gestionan de manera estructurada mediante componentes desacoplados, utilizando clases de modelo, servicios especializados y controladores de eventos.

Finalmente, el propósito central de este proyecto trasciende la funcionalidad operativa; busca validar la aplicabilidad práctica y el rendimiento de estructuras de datos avanzadas, tales como árboles para la organización jerárquica, pilas para el historial de operaciones y grafos para el cálculo de rutas óptimas, dentro de un entorno de software empresarial real.

4. Organización del proyecto

El proyecto se encuentra organizado en paquetes, lo cual permite separar responsabilidades y mantener una estructura clara. La organización principal del código fuente es la siguiente:

Figura 1. Estructura del proyecto en NetBeans



La figura muestra la organización del proyecto en paquetes, separando las estructuras de datos, modelos, servicios, excepciones, clase principal e interfaces gráficas.

4.1 Paquete estructuras

Este paquete contiene las estructuras de datos implementadas en el proyecto. En él se encuentran las clases relacionadas con el árbol binario de búsqueda, la pila de auditoría, el grafo logístico, la lista enlazada y los nodos auxiliares.

Clase	Descripción
ArbolBinarioBusqueda	Estructura utilizada para organizar y buscar productos.
NodoArbol	Nodo utilizado dentro del árbol binario.
ListaEnlazada	Estructura lineal dinámica utilizada como apoyo para almacenar elementos.
PilaAuditoria	Pila LIFO utilizada para registrar transacciones.
NodoPila	Nodo utilizado internamente por la pila.
GrafoLogistico	Grafo usado para representar almacenes y rutas logísticas.
Arista	Representa una conexión o ruta entre almacenes.

4.2 Paquete modelos

El paquete modelos contiene las clases que representan las entidades principales del sistema.

Clase	Descripción
Producto	Representa un producto del inventario.
Almacen	Representa un almacén dentro de la red logística.
Transaccion	Representa una operación o movimiento realizado en el inventario.

Estas clases permiten modelar la información básica que será procesada por los servicios y almacenada en las estructuras de datos.

4.3 Paquete servicios

El paquete servicios contiene la lógica principal del sistema.

Clase	Descripción
InventarioService	Gestiona productos, stock y transacciones.
LogisticaService	Gestiona almacenes, rutas logísticas y cálculo de caminos.

Estas clases sirven como intermediarias entre la interfaz gráfica y las estructuras de datos. De esta forma, los formularios no manipulan directamente las estructuras internas, sino que solicitan operaciones a los servicios.

4.4 Paquete excepciones

El paquete de excepciones contiene clases utilizadas para controlar errores específicos del sistema.

Excepción	Situación en la que se utiliza
CodigoProductoDuplicadoException	Cuando se intenta registrar un producto con un código ya existente.
ProductoNoEncontradoException	Cuando se intenta buscar u operar con un producto inexistente.
StockInsuficienteException	Cuando se intenta retirar una cantidad mayor al stock disponible.
FechaVencimientoInvalidaException	Cuando se ingresa una fecha de vencimiento inválida.
AlmacenNoEncontradoException	Cuando se intenta operar con un almacén no registrado.

El uso de excepciones personalizadas permite que los errores sean más claros y fáciles de manejar.

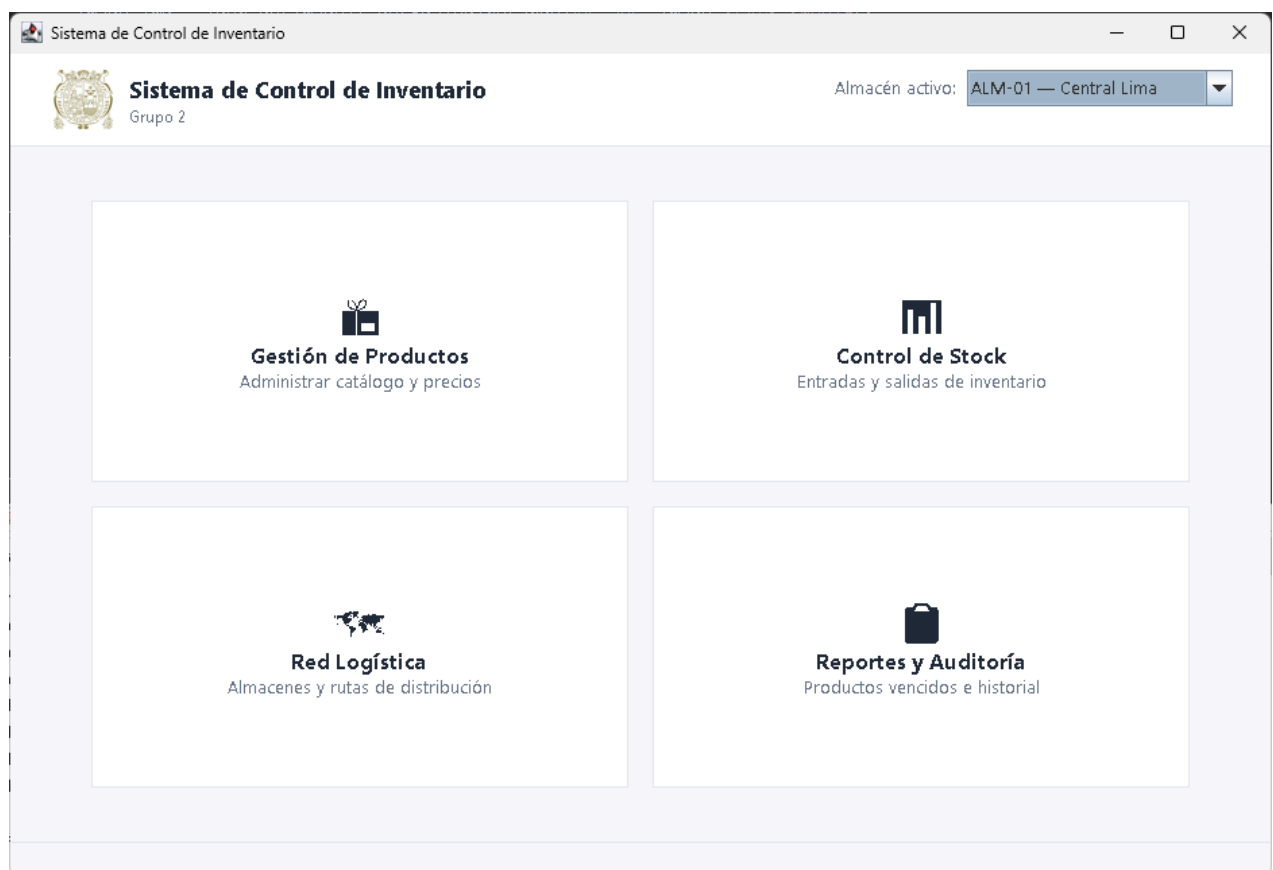
5. Descripción de módulos del sistema

El sistema cuenta con diferentes módulos gráficos que permiten al usuario interactuar con sus funcionalidades principales.

5.1 Módulo principal

El formulario principal permite acceder a los diferentes módulos del sistema. Desde esta ventana el usuario puede abrir la gestión de productos, el control de stock, la logística y los reportes.

Figura 2. Ventana principal del sistema



Descripción sugerida debajo de la imagen:

La ventana principal permite acceder a los módulos de gestión de productos, control de stock, logística y reportes.

5.2 Módulo de gestión de productos

El módulo de gestión de productos permite registrar nuevos productos, buscar productos existentes, modificar información y eliminar registros. Internamente, los productos se organizan utilizando una estructura de árbol binario de búsqueda, lo que facilita su administración mediante el código del producto.

Figura 3. Módulo de gestión de productos

The screenshot shows a web application window titled "Gestión de Productos". On the left is a sidebar with the heading "Datos del Producto". It contains several input fields: "Tipo:" with a dropdown menu set to "GENERAL"; "Categoría:" with a dropdown menu set to "Electronico"; "Código:" with a text input field containing "ELEC-003"; "Nombre:" with an empty text input field; "Precio (\$/):" with an empty text input field; "Stock Inicial:" with an empty text input field; "Stock Mínimo:" with an empty text input field; "Marca:" with an empty text input field; and "Vencimiento:" with an empty text input field. Below these fields are three buttons: "Registrar" (green), "Eliminar" (red), and "Limpiar" (grey). The main area of the window features a filter dropdown "Filtrar por categoría:" set to "Todas". Below this is a table with the following columns: "Código", "Nombre", "Categoría", "Precio (\$/)", "Stock Actual", "Stock Mín.", and "Fecha Venc.". The table contains 15 rows of product data. At the bottom of the main area is a large, empty grey rectangular box.

Código	Nombre	Categoría	Precio (\$/)	Stock Actual	Stock Mín.	Fecha Venc.
ALIM-001	Yogurt Gloria x6	Alimento	S/ 18.50	170	30	2026-07-23
ALIM-002	Aceite Primor 1L	Alimento	S/ 9.80	270	50	2028-01-08
ALIM-003	Leche Evaporada ...	Alimento	S/ 98.40	80	20	2027-07-08
ELEC-001	Laptop Lenovo Ide...	Electronico	S/ 2500.00	70	10	-
ELEC-002	Monitor Samsung ...	Electronico	S/ 850.00	25	8	-
FARM-001	Paracetamol 500m...	Farmaceutico	S/ 12.00	700	100	2028-07-08
FARM-002	Vacuna Influenza (...)	Farmaceutico	S/ 45.00	8	20	2027-01-08
HERRA-001	Taladro Inalambri...	Herramienta	S/ 459.00	3	5	-
HOGA-001	Silla Oficina Ergon...	Hogar/Oficina	S/ 380.00	2	10	-
ROPE-001	Polo Deportivo Ad...	Ropa	S/ 89.90	185	30	-

Descripción sugerida debajo de la imagen:

El módulo de gestión de productos permite registrar y administrar los productos del inventario. Los productos son organizados internamente mediante una estructura de árbol binario de búsqueda.

5.3 Módulo de control de stock

El módulo de control de stock permite registrar entradas y salidas de productos. Cada movimiento realizado genera una transacción, la cual se almacena en la pila de auditoría del sistema.

Esto permite llevar un historial de movimientos, donde la operación más reciente queda disponible en el tope de la pila.

Figura 4. Módulo de control de stock

Control de Stock

Filtrar por categoría:

Todas

Código	Nombre	Categoría	Stock Actual
ALIM-001	Yogurt Gloria x6	Alimento	170
ALIM-002	Aceite Primor 1L	Alimento	270
ALIM-003	Leche Evaporada Gloria x24	Alimento	80
ELEC-001	Laptop Lenovo IdeaPad	Electronico	70
ELEC-002	Monitor Samsung 24"	Electronico	25
FARM-001	Paracetamol 500mg x100	Farmaceutico	700
FARM-002	Vacuna Influenza (dosis)	Farmaceutico	8
HERRA-001	Taladro Inalambrico Bosch	Herramienta	3
HOGA-001	Silla Oficina Ergonomica	Hogar/Oficina	2
ROPE-001	Polo Deportivo Adidas	Ropa	185

Registrar Movimiento — Almacén: ALM-01

Código:

Cantidad:

Observación:

+ Entrada

- Salida

El módulo de control de stock permite registrar movimientos de inventario. Cada operación genera una transacción que se almacena en la pila de auditoría.

5.4 Módulo de logística

El módulo de logística permite administrar almacenes y rutas entre ellos. Los almacenes se representan como vértices de un grafo y las rutas como aristas con peso, donde el peso corresponde a la distancia en kilómetros.

Este módulo también permite calcular la ruta más corta entre dos almacenes mediante el algoritmo de Dijkstra.

Figura 5. Módulo de logística

El módulo de logística permite registrar almacenes y conectarlos mediante rutas. Internamente se utiliza un grafo ponderado para representar la red logística.

5.5 Módulo de reportes

El módulo de reportes permite visualizar información generada por el sistema, como productos registrados, movimientos de inventario, historial de transacciones o información de la red logística.

Figura 6. Módulo de reportes

Reportes y Auditoría

Inventario Completo

Productos Perecibles Vencidos

Historial de Movimientos

Código	Nombre	Categoría	Tipo	Precio (S/)	Stock	Stock Mín.	Marca	Fecha Venc.
ALIM-001	Yogurt Gloria x6	Alimento	PERECIBLE	S/ 18.50	170	30	N/A	2026-07-23
ALIM-002	Aceite Primor 1L	Alimento	PERECIBLE	S/ 9.80	270	50	N/A	2028-01-08
ALIM-003	Leche Evaporada GL...	Alimento	PERECIBLE	S/ 98.40	80	20	N/A	2027-07-08
ELEC-001	Laptop Lenovo Ide...	Electronico	GENERAL	S/ 2500.00	70	10	Lenovo	-
ELEC-002	Monitor Samsung ...	Electronico	GENERAL	S/ 850.00	25	8	Samsung	-
FARM-001	Paracetamol 500m...	Farmaceutico	PERECIBLE	S/ 12.00	700	100	N/A	2028-07-08
FARM-002	Vacuna Influenza (...)	Farmaceutico	PERECIBLE	S/ 45.00	8	20	N/A	2027-01-08
HERRA-001	Taladro Inalambric...	Herramienta	GENERAL	S/ 459.00	3	5	Bosch	-
HOGA-001	Silla Oficina Ergono...	Hogar/Oficina	GENERAL	S/ 380.00	2	10	Actiu	-
ROPE-001	Polo Deportivo Adi...	Ropa	GENERAL	S/ 89.90	185	30	Adidas	-

Actualizar

El módulo de reportes permite consultar información del inventario, movimientos registrados y datos relevantes de la red logística.

6. Estructuras de datos implementadas

El proyecto utiliza diversas estructuras de datos para resolver problemas específicos. Las estructuras principales son la lista enlazada, el árbol binario de búsqueda, la pila y el grafo.

6.1 Lista enlazada como estructura auxiliar

La lista enlazada es una estructura de datos lineal que permite almacenar elementos de forma dinámica. A diferencia de un arreglo tradicional, una lista enlazada no requiere definir un tamaño fijo desde el inicio.

En el proyecto, la clase ListaEnlazada cumple un rol auxiliar importante, ya que se utiliza para almacenar colecciones internas, como listas de vértices, aristas, caminos o transacciones.

Su uso permite que otras estructuras del sistema funcionen sin depender directamente de colecciones predefinidas para sus operaciones principales.

Aplicación en el sistema

La lista enlazada se utiliza como apoyo en diferentes partes del proyecto:

Uso	Descripción
Grafo logístico	Almacena los vértices y las listas de aristas.
Resultado de Dijkstra	Almacena el camino encontrado entre almacenes.
Pila de auditoría	Puede convertir las transacciones a una lista para mostrarlas sin modificar la pila.
Reportes	Permite recorrer información almacenada en estructuras internas.

6.2 Árbol Binario de Búsqueda

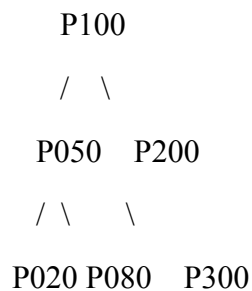
El Árbol Binario de Búsqueda, también conocido como ABB o BST, es una estructura de datos jerárquica compuesta por nodos. Cada nodo puede tener un hijo izquierdo y un hijo derecho.

En este proyecto, el árbol binario de búsqueda se utiliza para organizar los productos del inventario. La clave principal de comparación es el código del producto, ya que este permite identificar de forma única cada registro.

La propiedad principal del árbol binario de búsqueda es la siguiente:

- Los elementos menores se ubican en el subárbol izquierdo.
- Los elementos mayores se ubican en el subárbol derecho.

Representación conceptual



En este ejemplo, si se desea buscar el producto P080, el sistema no necesita recorrer todos los productos. Primero compara con la raíz, luego decide si debe continuar por la izquierda o por la derecha.

Aplicación dentro del sistema

El árbol binario de búsqueda permite realizar operaciones importantes sobre productos, tales como:

- Registrar productos.
- Buscar productos por código.
- Evitar códigos duplicados.
- Recorrer productos de manera ordenada.
- Apoyar la generación de reportes del inventario.

Gracias a esta estructura, el sistema puede organizar los productos de forma más eficiente que una lista lineal, especialmente cuando el número de productos aumenta.

Código 1. Fragmento del Árbol Binario de Búsqueda

```
public void insertar(Producto producto) {  
    // [REQUISITO RUBRICA: ÁRBOL BINARIO DE BÚSQUEDA] - inicio de inserción  
    this.raiz = insertarRec(this.raiz, producto);  
    this.tamanio++;  
}  
  
public Producto buscar(String codigo) {  
    // [REQUISITO RUBRICA: ÁRBOL BINARIO DE BÚSQUEDA] - inicio de búsqueda  
    NodoArbol resultado = buscarRec(this.raiz, codigo);  
    if (resultado == null) {  
        throw new ProductoNoEncontradoException(codigo);  
    }  
    return resultado.dato;  
}  
  
public ListaEnlazada<Producto> recorrerPreorden() {  
    ListaEnlazada<Producto> lista = new ListaEnlazada<>();  
    preordenRec(this.raiz, lista);  
    return lista;  
}
```

Complejidad del Árbol Binario de Búsqueda

Operación	Complejidad promedio	Peor caso
Inserción	$O(\log n)$	$O(n)$
Búsqueda	$O(\log n)$	$O(n)$
Eliminación	$O(\log n)$	$O(n)$
Recorrido completo	$O(n)$	$O(n)$

El peor caso ocurre cuando el árbol queda desbalanceado, por ejemplo, si los productos se insertan en orden ascendente o descendente. En ese escenario, el árbol puede comportarse de forma similar a una lista enlazada.

6.3 Pila de Auditoría

La pila es una estructura de datos lineal que funciona bajo el principio LIFO, cuyas siglas significan “Last In, First Out”, es decir, “último en entrar, primero en salir”.

En el proyecto, la pila se implementa mediante la clase PilaAuditoria, utilizando nodos enlazados representados por la clase NodoPila. Esta pila almacena objetos de tipo Transaccion, que representan movimientos realizados dentro del inventario.

Principio LIFO

La pila permite que la última transacción registrada sea la primera en consultarse o retirarse. Esto resulta adecuado para un sistema de auditoría, porque normalmente se requiere revisar primero la operación más reciente.

Representación conceptual:

Tope

↓

[Transacción 3] ← última operación registrada

[Transacción 2]

[Transacción 1] ← primera operación registrada

Si se ejecuta una operación de extracción, se retira primero la Transacción 3, porque se encuentra en el tope.

Implementación de la pila

La pila mantiene una referencia al nodo superior, llamado tope. Cada nodo contiene una transacción y una referencia al siguiente nodo.

tope → [Transacción reciente] → [Transacción anterior] → [Transacción inicial] → null

Cuando la pila está vacía, el valor de tope es null.

Operaciones principales

Operación	Descripción	Complejidad
push	Agrega una nueva transacción al tope.	O(1)
pop	Extrae la transacción ubicada en el tope.	O(1)
peek	Consulta el tope sin eliminarlo.	O(1)
estaVacía	Verifica si la pila está vacía.	O(1)
toList	Convierte la pila en lista para visualizarla.	O(n)
vaciar	Elimina las referencias internas de la pila.	O(1)

Código 2. Fragmento de la Pila de Auditoría

```

public void push(Transaccion transaccion) {
    // [REQUISITO RUBRICA: PILAS] - Operación PUSH: agrega al tope
    NodoPila nuevoNodo = new NodoPila(transaccion); // Paso 1: crear nodo
    nuevoNodo.siguiente = this.tope;                // Paso 2: enlazar con el tope actual
    this.tope = nuevoNodo;                          // Paso 3: el nuevo nodo ES el nuevo tope
    this.tamano++;
}

```

Código 3. Fragmento de extracción o consulta de la pila

```

    */
    public Transaccion pop() {
        // [REQUISITO RUBRICA: PILAS] - Operación POP: extrae del tope
        if (estaVacia()) {
            throw new IllegalStateException(
                "No se puede desapilar: la pila de auditoria esta vacia.");
        }
        Transaccion dato = this.tope.dato;          // Paso 1: guardar dato del tope
        this.tope = this.tope.siguiente;          // Paso 2: mover tope al nodo anterior
        this.tamanio--;
        return dato;
    }

    public Transaccion peek() {
        // [REQUISITO RUBRICA: PILAS] - Operación PEEK: consulta sin modificar la pila
        if (estaVacia()) {
            throw new IllegalStateException(
                "No se puede consultar: la pila de auditoria esta vacia.");
        }
        return this.tope.dato;
    }
}
```

El método permite obtener la transacción más reciente. En el caso de pop, la transacción se extrae de la pila; en el caso de peek, solo se consulta sin modificar la estructura.

Aplicación de la pila en el sistema

La pila de auditoría se utiliza para registrar los movimientos de inventario. Por ejemplo, cuando se registra una entrada o salida de stock, se genera una transacción y esta se apila.

Esto permite mantener un historial de operaciones en orden cronológico inverso, mostrando primero la operación más reciente.

Ventajas de usar una pila

El uso de una pila es adecuado porque:

- Permite registrar rápidamente nuevas transacciones.
- Facilita consultar la operación más reciente.
- Tiene operaciones principales de complejidad constante.
- Se adapta naturalmente al concepto de historial reciente.

6.4 Grafo Logístico

El grafo es una estructura de datos que permite representar relaciones entre elementos. En el proyecto, el grafo se utiliza para representar la red logística de almacenes.

La clase GrafoLogistico implementa un grafo no dirigido y ponderado mediante una lista de adyacencia. Cada vértice representa un almacén y cada arista representa una ruta entre dos almacenes.

Elementos del grafo

Elemento	Representación en el sistema
Vértice	Un almacén.
Arista	Una ruta entre dos almacenes.
Peso	Distancia en kilómetros entre almacenes.
Lista de adyacencia	Lista de rutas conectadas a cada almacén.

Grafo no dirigido

El grafo es no dirigido porque una conexión entre dos almacenes se registra en ambos sentidos.

Por ejemplo, si se conecta el almacén A con el almacén B, el sistema registra:

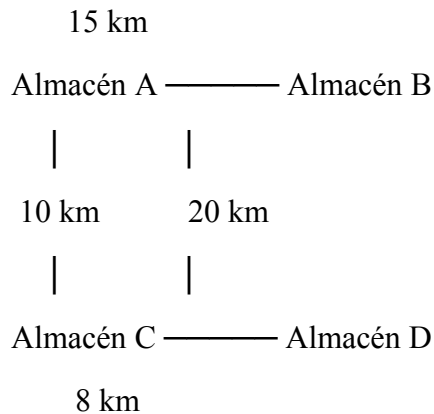
- $A \rightarrow B$
- $B \rightarrow A$

Esto representa que la ruta puede recorrerse desde A hacia B y también desde B hacia A.

Grafo ponderado

El grafo es ponderado porque cada ruta posee un peso. En este caso, el peso representa la distancia en kilómetros.

Ejemplo conceptual:



Gracias a los pesos, el sistema puede calcular la ruta más corta entre dos almacenes.

Lista de adyacencia

El grafo utiliza una lista de adyacencia. Esto significa que cada almacén mantiene una lista de sus conexiones directas.

Ejemplo:

A: B(15 km), C(10 km)

B: A(15 km), D(20 km)

C: A(10 km), D(8 km)

D: B(20 km), C(8 km)

Esta forma de representación es adecuada porque no todos los almacenes necesitan estar conectados entre sí.

Código 4. Fragmento de conexión entre almacenes

```
public void conectar(String idOrigen, String idDestino, double distanciaKm, String descripcion) {
    VerticeLogistico vOrigen = obtenerVertice(idOrigen);
    VerticeLogistico vDestino = obtenerVertice(idDestino);

    if (vOrigen == null) throw new AlmacenNoEncontradoException(idOrigen);
    if (vDestino == null) throw new AlmacenNoEncontradoException(idDestino);

    vOrigen.aristas.add(new Arista(idDestino, distanciaKm, descripcion));
    vDestino.aristas.add(new Arista(idOrigen, distanciaKm, descripcion));

    this.numeroAristas++;
}
```

El fragmento muestra cómo se conecta un almacén origen con un almacén destino. La ruta se agrega en ambos sentidos, lo que demuestra que el grafo es no dirigido. Además, se almacena un peso asociado a la distancia en kilómetros.

Operaciones principales del grafo

Operación	Descripción
agregarAlmacen	Agrega un nuevo almacén como vértice.
conectar	Crea una ruta entre dos almacenes.
eliminarAlmacen	Elimina un almacén y sus rutas asociadas.
eliminarRuta	Elimina una conexión entre dos almacenes.
mostrarRed	Muestra la red logística como lista de adyacencia.
listarAlmacenes	Muestra los almacenes registrados.
existeAlmacen	Verifica si un almacén existe.
dijkstra	Calcula la ruta más corta entre almacenes.

Figura 7. Representación de la red logística

```
public String mostrarRed() {
    if (vertices.isEmpty()) return " La red logistica esta vacia.";

    StringBuilder sb = new StringBuilder();
    sb.append(" RED LOGISTICA - LISTA DE ADYACENCIA\n");
    sb.append(" =====\n");

    for (VerticeLogistico v : vertices) {
        sb.append(String.format(" > [%s] %s | %s | Cap: %d ud.\n",
            v.almacen.getId(), v.almacen.getNombre(), v.almacen.getTipo(), v.almacen.getCapacidadMaxima()));

        if (v.aristas.isEmpty()) {
            sb.append(" (sin conexiones)\n");
        } else {
            for (Arista a : v.aristas) {
                Almacen dest = getAlmacen(a.getIdDestino());
                sb.append(String.format(" %s [%s]\n", a, dest != null ? dest.getNombre() : "?"));
            }
        }
    }
    sb.append(String.format("\n Vertices: %d | Aristas: %d\n", vertices.size(), numeroAristas));
    return sb.toString();
}
```

La red logística se representa mediante un grafo, donde los almacenes son vértices y las rutas son aristas ponderadas por distancia.

6.5 Algoritmo de Dijkstra

El algoritmo de Dijkstra se utiliza para encontrar la ruta más corta entre dos vértices de un grafo ponderado con pesos no negativos. En este proyecto, se aplica para calcular la ruta logística óptima entre dos almacenes.

El peso de cada arista representa la distancia en kilómetros entre almacenes. Por ello, el algoritmo permite determinar cuál es el camino con menor distancia total.

Funcionamiento general

El algoritmo realiza los siguientes pasos:

1. Verifica que el almacén de origen exista.
2. Verifica que el almacén de destino exista.
3. Inicializa las distancias hacia todos los almacenes con un valor muy grande.
4. Asigna distancia cero al almacén de origen.
5. Selecciona el almacén no visitado con menor distancia acumulada.
6. Recorre sus vecinos o almacenes conectados.
7. Actualiza las distancias si encuentra un camino más corto.

8. Guarda los predecesores para reconstruir la ruta final.
9. Devuelve el camino encontrado, la distancia total y si el destino es alcanzable.

Datos utilizados por el algoritmo

Dato	Función
ids	Almacena los identificadores de los almacenes.
distancias	Guarda la distancia mínima conocida desde el origen.
predecesores	Permite reconstruir el camino final.
visitados	Indica qué almacenes ya fueron procesados.

Código 5. Fragmento del algoritmo de Dijkstra

```
for (Arista arista : vActual.aristas) {  
    String idVecino = arista.getIdDestino();  
    int v = buscarIndice(ids, idVecino);  
  
    if (v != -1 && !visitados[v] && distancias[u] != Double.MAX_VALUE) {  
        double nuevaDist = distancias[u] + arista.getPeso();  
        if (nuevaDist < distancias[v]) {  
            distancias[v] = nuevaDist;  
            predecesores[v] = ids[u];  
        }  
    }  
}
```

El fragmento corresponde al proceso de relajación de Dijkstra. Si se encuentra una distancia menor hacia un almacén vecino, se actualiza la distancia mínima y se guarda el predecesor correspondiente.

Resultado de Dijkstra

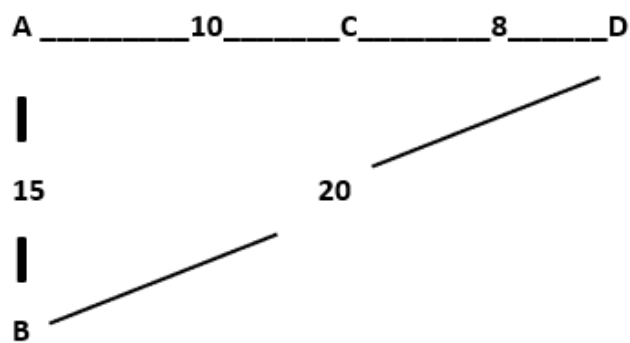
El método de Dijkstra devuelve un resultado que contiene:

Elemento	Descripción
----------	-------------

Camino	Lista con los almacenes que forman la ruta óptima.
Distancia total	Suma de las distancias de las rutas recorridas.
Alcanzable	Indica si existe una ruta entre origen y destino.

Ejemplo conceptual

Supongamos la siguiente red:



Para ir de A hacia D, existen varias rutas posibles:

Ruta 1: $A \rightarrow B \rightarrow D = 15 + 20 = 35$ km

Ruta 2: $A \rightarrow C \rightarrow D = 10 + 8 = 18$ km

La ruta óptima sería:

$A \rightarrow C \rightarrow D = 18$ km

Figura 8. Resultado del cálculo de ruta óptima

The screenshot displays a web interface for route optimization. At the top, a section titled "Optimización de Rutas Logísticas" contains two dropdown menus: "Origen:" set to "ALM-02 — Hub Callao" and "Destino:" set to "ALM-03 — Regional SJL". Below these is a blue button labeled "Calcular Ruta Óptima". The results are shown in a green box titled "Resultado de Ruta Óptima", which contains the text: "Ruta Óptima Encontrada", "Recorrido: ALM-02 -> ALM-01 -> ALM-03", and "Distancia Total: 37.50 km".

El sistema calcula la ruta más corta entre dos almacenes utilizando el algoritmo de Dijkstra, considerando la distancia en kilómetros como peso de cada ruta.

Complejidad de Dijkstra

La implementación utilizada en el proyecto trabaja con arreglos y estructuras personalizadas. Al seleccionar el vértice no visitado con menor distancia mediante recorrido lineal, la complejidad aproximada es:

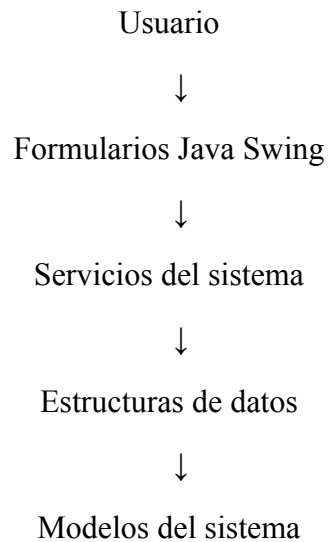
$$O(V^2)$$

Donde V representa la cantidad de almacenes o vértices del grafo.

Esta implementación es adecuada para redes logísticas pequeñas o medianas y permite demostrar claramente el funcionamiento del algoritmo sin depender de estructuras externas como una cola de prioridad.

7. Funcionamiento general del sistema

El funcionamiento general del sistema puede representarse como una secuencia de interacción entre el usuario, los formularios gráficos, los servicios y las estructuras de datos.



El usuario interactúa con la aplicación mediante formularios. Estos formularios envían las solicitudes a las clases de servicio, las cuales procesan la lógica correspondiente. Luego, los servicios utilizan las estructuras de datos para almacenar, buscar, actualizar o consultar la información.

7.1 Registro de productos

Cuando el usuario registra un nuevo producto, el sistema valida los datos ingresados y luego almacena el producto en la estructura correspondiente. Si el código ya existe, se lanza una excepción personalizada para evitar duplicados.

Figura 9. Registro exitoso de producto

Datos del Producto

Tipo:

Categoría:

Código:

Nombre:

Precio (S/):

Stock Inicial:

Stock Mínimo:

Marca:

Vencimiento:

Éxito

 Producto registrado con éxito.

Filtrar por categoría:

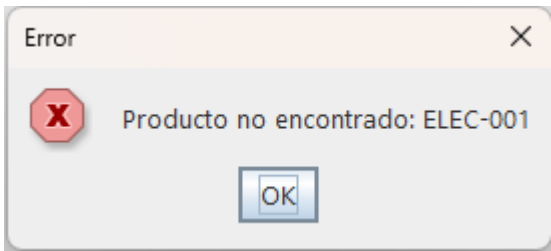
Código	Nombre	Categoría	Precio (S/)	Stock Actual	Stock Mín.	Fecha Venc.
ROPE-001	Polo Deportivo Adidas	Ropa	S/ 89.90	185	30	-

La figura muestra el registro exitoso de un producto en el sistema. El producto queda disponible para búsquedas, movimientos de stock y reportes.

7.2 Búsqueda de productos

La búsqueda de productos se realiza mediante el código del producto. La estructura de árbol binario de búsqueda permite ubicar el producto comparando códigos y recorriendo solo las ramas necesarias.

Figura 10. Búsqueda de producto



7.3 Movimiento de stock

Cuando se realiza una entrada o salida de stock, el sistema actualiza la cantidad disponible del producto. Además, genera una transacción y la almacena en la pila de auditoría.

Figura 11. Registro de movimiento de stock

ALIMENTO	ACCIONES PRODUCTO	ALIMENTO	CANTIDAD
ALIM-003	Leche Evaporada Gloria x24	Alimento	80
ELEC-001	Laptop Lenovo IdeaPad	Electronico	70

Registrar Movimiento — Almacén: ALM-01

Código: Cantidad: Observación:

+ Entrada

- Salida

Éxito

Entrada registrada correctamente.

OK

ALIM-003	Leche Evaporada Gloria x24	Alimento	120
----------	----------------------------	----------	-----

Cada movimiento de stock actualiza la cantidad disponible del producto y genera una transacción que queda registrada en la pila de auditoría.

7.4 Registro de almacenes y rutas

El usuario puede registrar almacenes dentro de la red logística. Posteriormente, puede conectar dos almacenes mediante una ruta con distancia en kilómetros.

Figura 12. Registro de almacenes y rutas

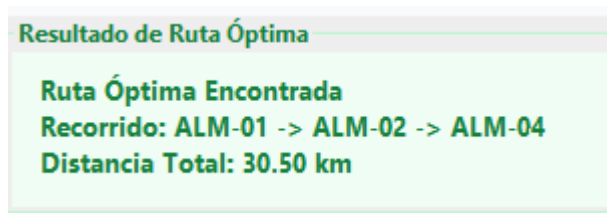
Red Logística Actual	
RED LOGISTICA - LISTA DE ADYACENCIA	
<pre>► [ALM-01] Central Lima CENTRAL Cap: 50,000 ud. --(12.5 km)-> [ALM-02] via Av. Colonial [Hub Callao] --(25.0 km)-> [ALM-03] via Av. Universitaria [Regional SJL] --(1232.0 km)-> [ALM-01] via aa [Central Lima] --(1232.0 km)-> [ALM-01] via aa [Central Lima] --(111.0 km)-> [ALM-01] via aaa [Central Lima] --(111.0 km)-> [ALM-01] via aaa [Central Lima] ► [ALM-02] Hub Callao HUB Cap: 30,000 ud. --(12.5 km)-> [ALM-01] via Av. Colonial [Central Lima] --(18.0 km)-> [ALM-04] via Panamericana Sur [Tienda Surco] --(10.0 km)-> [ALM-06] via Av. Arequipa [Tienda Miraflores] ► [ALM-03] Regional SJL REGIONAL Cap: 20,000 ud. --(25.0 km)-> [ALM-01] via Av. Universitaria [Central Lima] --(30.0 km)-> [ALM-04] via Via Expresa [Tienda Surco] ► [ALM-04] Tienda Surco TIENDA Cap: 5,000 ud. --(18.0 km)-> [ALM-02] via Panamericana Sur [Hub Callao] --(30.0 km)-> [ALM-03] via Via Expresa [Regional SJL] --(55.0 km)-> [ALM-05] via Panamericana Sur [Regional Ica] ► [ALM-05] Regional Ica REGIONAL Cap: 15,000 ud. --(55.0 km)-> [ALM-04] via Panamericana Sur [Tienda Surco] --(80.0 km)-> [ALM-06] via Carretera Ica-Lima [Tienda Miraflores] ► [ALM-06] Tienda Miraflores TIENDA Cap: 3,000 ud. --(80.0 km)-> [ALM-05] via Carretera Ica-Lima [Regional Ica] --(10.0 km)-> [ALM-02] via Av. Arequipa [Hub Callao]</pre>	
Vertices: 6 Aristas: 9	

Los almacenes se representan como vértices del grafo, mientras que las rutas se representan como aristas con peso.

7.5 Cálculo de ruta logística

Para calcular la mejor ruta entre dos almacenes, el usuario selecciona un origen y un destino. El sistema aplica Dijkstra y devuelve el camino más corto junto con la distancia total.

Figura 13. Ejecución del cálculo de ruta logística



El sistema muestra la ruta óptima entre almacenes y la distancia total calculada mediante el algoritmo de Dijkstra.

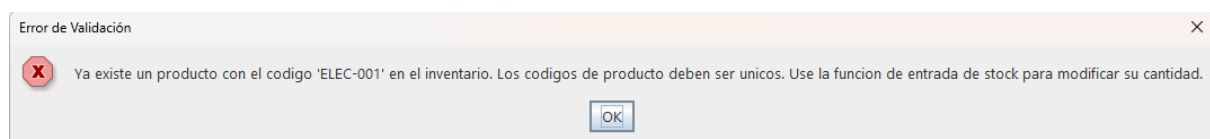
8. Manejo de excepciones

El sistema utiliza excepciones personalizadas para controlar errores específicos y mostrar mensajes más claros al usuario.

El manejo de excepciones permite evitar que el sistema falle de forma inesperada. En lugar de eso, se controla la situación y se informa al usuario sobre el error ocurrido.

Excepción	Descripción
CodigoProductoDuplicadoException	Evita registrar dos productos con el mismo código.
ProductoNoEncontradoException	Informa que el producto buscado no existe.
StockInsuficienteException	Evita retirar más unidades de las disponibles.
FechaVencimientoInvalidaException	Controla fechas de vencimiento no válidas.
AlmacenNoEncontradoException	Informa que un almacén no existe en la red logística.

Figura 14. Mensaje de error controlado



El sistema utiliza excepciones personalizadas para controlar errores específicos y mostrar mensajes comprensibles al usuario.

9. Evaluación técnica del proyecto

El proyecto presenta una estructura organizada y modular, lo cual facilita su mantenimiento. La separación en paquetes permite identificar claramente las responsabilidades de cada clase.

El paquete estructuras contiene las implementaciones principales de estructuras de datos, mientras que el paquete modelos representa las entidades del sistema. Por otro lado, el paquete servicios concentra la lógica de negocio, y los formularios gráficos permiten la interacción con el usuario.

Desde el punto de vista técnico, el uso de estructuras de datos personalizadas representa uno de los aspectos más importantes del proyecto. El árbol binario de búsqueda permite organizar productos, la pila de auditoría permite registrar movimientos recientes de inventario y el grafo logístico permite representar almacenes y rutas.

Además, el algoritmo de Dijkstra permite resolver un problema real de logística: encontrar la ruta más corta entre dos almacenes. Esto demuestra la aplicación práctica de algoritmos de grafos dentro de un sistema funcional.

El manejo de excepciones personalizadas también fortalece el sistema, ya que permite controlar errores específicos y mejorar la experiencia del usuario.

10. Posibles mejoras

Aunque el sistema cumple con los objetivos planteados, se pueden considerar algunas mejoras futuras:

Mejora	Descripción
Persistencia de datos	Guardar productos, almacenes y transacciones en archivos o base de datos.
Validación de rutas duplicadas	Evitar registrar dos veces la misma conexión entre almacenes.
Validación de distancias	Asegurar que las rutas tengan distancias positivas.
Balanceo del árbol	Implementar un árbol AVL o rojo-negro para mejorar el rendimiento en el peor caso.
Mejora visual de la interfaz	Optimizar el diseño gráfico de los formularios.
Pruebas unitarias	Agregar pruebas automatizadas para validar estructuras y servicios.
Exportación de reportes	Permitir exportar reportes en PDF, Excel u otros formatos.
Gestión de usuarios	Agregar roles como administrador, operador o supervisor.

Estas mejoras permitirían que el sistema sea más robusto, escalable y útil en un entorno real.

11. Conclusiones

El sistema desarrollado consolida la aplicación práctica de conceptos fundamentales de la ciencia de la computación y la ingeniería de software en un entorno real de gestión de inventarios y optimización logística. A nivel algorítmico, el proyecto destaca por la implementación de un Árbol Binario de Búsqueda empleado para la organización jerárquica de los productos, lo que optimiza los tiempos de búsqueda y recuperación de información mediante códigos indexados. Asimismo, se diseñó una Pila de Auditoría bajo el principio LIFO (*Last-In, First-Out*), permitiendo un rastreo histórico y ordenado que prioriza la visualización de las operaciones más recientes, mientras que un Grafo Logístico, respaldado por el algoritmo de Dijkstra, se utilizó para modelar la red de distribución y calcular analíticamente el camino más corto entre los diferentes almacenes.

En el ámbito de la arquitectura de software, el sistema evidencia un diseño robusto mediante el paradigma de Programación Orientada a Objetos, aplicando una estricta separación de responsabilidades a través de paquetes que delimitan las capas de modelo, servicio e interfaz. Además, la estabilidad y la experiencia de usuario se reforzaron mediante la integración de un sistema de control de flujo basado en excepciones personalizadas y una interfaz gráfica de escritorio interactiva desarrollada con la librería Java Swing.

Finalmente, el proyecto no solo cumple con los requerimientos operativos planteados, sino que valida la viabilidad de integrar estructuras de datos avanzadas en el desarrollo de software empresarial. Asimismo, la ejecución del proyecto bajo un enfoque colaborativo fortaleció la gestión del trabajo en equipo, la distribución estratégica de responsabilidades y la correcta integración y acoplamiento de componentes dentro de una solución tecnológica unificada.

12. Bibliografía

- Cormen, T., Leiserson, C., Rivest, R. y Stein, C. *Introduction to Algorithms*.
- Deitel, P. y Deitel, H. *Java: How to Program*.
- Weiss, M. *Data Structures and Algorithm Analysis in Java*.
- Documentación oficial de Java.
- Material del curso sobre estructuras de datos, árboles, pilas y grafos.

13. Anexos

Anexo 1. Fragmentos de código relevantes

En esta sección se pueden colocar fragmentos adicionales de código que no se incluyeron en el cuerpo principal del informe.

Anexo 1.1 Código del Árbol Binario de Búsqueda

```
public void insertar(Producto producto) { 3 usages
    this.raiz = insertarRec(this.raiz, producto);
    this.tamanio++;
}
```

```
public Producto buscar(String codigo) { 6 usages
    NodoArbol resultado = buscarRec(this.raiz, codigo);
    if (resultado == null) {
        throw new ProductoNoEncontradoException(codigo);
    }
    return resultado.dato;
}
```

```
public void eliminar(String codigo) { 3 usages
    if (!existe(codigo)) throw new ProductoNoEncontradoException(codigo);
    this.raiz = eliminarRec(this.raiz, codigo);
    this.tamanio--;
}
```

Anexo 1.2 Código de la Pila de Auditoría

```
public void push(Transaccion transaccion) { 5 usages
    NodoPila nuevoNodo = new NodoPila(transaccion); // Crear nodo
    nuevoNodo.siguiente = this.tope; // Enlazar nodo
    this.tope = nuevoNodo; // Actualizar tope
    this.tamanio++;
}
```

```
public Transaccion pop() { 2 usages
    if (estaVacia()) {
        throw new IllegalStateException(
            "No se puede desapilar: la pila de auditoria esta vacia.");
    }
    Transaccion dato = this.tope.dato;          // guardar dato del tope
    this.tope = this.tope.siguiete;           // mover tope al nodo anterior
    this.tamano--;
    return dato;
}
```